

Elaborato: Implementazione Hardware di un Gate di Hadamard

Corso: Introduzione al trattamento quantistico dei dati

Studente: Ildo Tiberio [0322500009]

Obiettivo: Progettazione, implementazione e simulazione in linguaggio VHDL dell'operatore quantistico di Hadamard su target FPGA (Boolean Board).

1. Descrizione del Progetto e Scelte Architettureali

L'elaborato si focalizza sulla realizzazione di un Gate di Hadamard per un singolo qubit.

L'operazione trasforma lo stato di ingresso secondo la matrice unitaria di Hadamard. Per la mappatura hardware è stata adottata l'aritmetica **Fixed-Point** con i seguenti vincoli costruttivi:

- **Lunghezza parola:** 12 bit con segno (signed).
- **Parte frazionaria:** 8 bit (formato Q3.8).
- **Costante di Hadamard ($1/\sqrt{2}$):** Convertita nel valore intero **181** (calcolato come $0.7071 \cdot 2^8$). Si nota che questa quantizzazione introduce un errore trascurabile (≈ 0.000075), perfettamente tollerabile per l'applicazione.

2. Listato del Codice RTL (Top Level)

Il design segue un approccio RTL sincrono. Di seguito il codice della *top entity*, ampiamente commentato per descrivere l'effetto pratico di ogni operazione sull'hardware sintetizzato.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity hadamard_top is
  Port (
    clk    : in  STD_LOGIC;
    res    : in  STD_LOGIC;
    alpha_in : in  STD_LOGIC_VECTOR(11 downto 0); -- Ampiezza di probabilità stato |0>
    beta_in  : in  STD_LOGIC_VECTOR(11 downto 0); -- Ampiezza di probabilità stato |1>
    alpha_out: out STD_LOGIC_VECTOR(11 downto 0);
    beta_out : out STD_LOGIC_VECTOR(11 downto 0)
  );
end hadamard_top;

architecture Behavioral of hadamard_top is
  -- Definisco la costante di Hadamard quantizzata per ottimizzare le risorse
  constant H_COEFF : signed(11 downto 0) := to_signed(181, 12);

  -- Segnali interni per i registri di campionamento (Pipelining)
  signal r_alpha, r_beta : signed(11 downto 0);

  -- Segnali a 24 bit per ospitare il risultato senza overflow durante la moltiplicazione
  signal mult_a, mult_b : signed(23 downto 0);
begin
  process(clk, res)
  begin
    if res = '1' then
      -- Reset asincrono per inizializzare i registri in uno stato noto
      r_alpha  <= (others => '0');
      r_beta   <= (others => '0');
      alpha_out <= (others => '0');
      beta_out  <= (others => '0');
    elsif rising_edge(clk) then
```

-- FASE 1: Campionamento Ingressi (Vincolo di progetto)

-- Assorbe i valori esterni nei registri interni della FPGA

```
r_alpha <= signed(alpha_in);
```

```
r_beta  <= signed(beta_in);
```

-- FASE 2: Calcolo Combinatorio (Mappato sui DSP)

-- Il coefficiente moltiplica entrambi gli stati in parallelo

```
mult_a <= H_COEFF * r_alpha;
```

```
mult_b <= H_COEFF * r_beta;
```

-- FASE 3: Somma, Normalizzazione e Registrazione delle Uscite

-- mult_a + mult_b modella la prima riga della matrice [1 1]

-- mult_a - mult_b modella la seconda riga della matrice [1 -1]

-- Lo shift_right di 8 posizioni divide per 256, riportando il

-- prodotto a 24 bit nel formato originale Q3.8 a 12 bit.

```
alpha_out <= std_logic_vector(resize(shift_right(mult_a + mult_b, 8), 12));
```

```
beta_out  <= std_logic_vector(resize(shift_right(mult_a - mult_b, 8), 12));
```

```
end if;
```

```
end process;
```

```
end Behavioral;
```

Hadamard_Gate_Project - [C:/Users/ildot/Documents/Vivado/Hadamard_Gate_Project/Hadamard_Gate_Project.xpr] - Vivado 2025.2

Implementation Complete ✓

Default Layout

Flow Navigator

PROJECT MANAGER

Sources

constrs_1

Simulation Sources (1)

sim_1 (1)

tb_hadamard(Behavioral) (tb_hadamard.vhd) (1)

Utility Sources

utl : hadamard_top(Behavioral) (hadamard_top.vhd)

Hierarchy Libraries Compile Order

Properties

tb_hadamard.vhd

Enabled

Location: C:/Users/ildot/Documents/Vivado/Hadamard_Gate_Project/Hadamard_

Type: VHDL

General Properties

Tcl Console Messages Log Reports Design Runs Linter

Violations (6) Waived (0) All (6)

Rule ID	RTL Name	RTL Hierarchy	Message Body	File Name
ASSIGN-5#1	mult_c	hadamard_top	Some bits in 'mult_c' are not set. First unset bit index is 0'.	hadamard_top.vhd : 43
ASSIGN-5#2	mult_d	hadamard_top	Some bits in 'mult_d' are not set. First unset bit index is 0'.	hadamard_top.vhd : 43
ASSIGN-5#3	sum_alpha	hadamard_top	Some bits in 'sum_alpha' are not set. First unset bit index is 0'.	hadamard_top.vhd : 44
ASSIGN-5#4	sum_beta	hadamard_top	Some bits in 'sum_beta' are not set. First unset bit index is 0'.	hadamard_top.vhd : 44

48:10 Insert VHDL

3. Listato del Test-bench e Analisi della Simulazione

Per validare la correttezza formale dell'operatore, è stato prodotto il seguente ambiente di simulazione (Test-bench), il quale inietta nel sistema 4 configurazioni di stato quantistico.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_hadamard is
end tb_hadamard;

architecture Behavioral of tb_hadamard is
    component hadamard_top
        Port ( clk, res : in STD_LOGIC;
              alpha_in, beta_in : in STD_LOGIC_VECTOR(11 downto 0);
              alpha_out, beta_out : out STD_LOGIC_VECTOR(11 downto 0));
    end component;

    signal clk, res : std_logic := '0';
    signal alpha_in, beta_in, alpha_out, beta_out : std_logic_vector(11 downto 0) := (others => '0');
    constant clk_period : time := 10 ns;
begin
    uut: hadamard_top port map (clk, res, alpha_in, beta_in, alpha_out, beta_out);

    clk_process : process begin
        clk <= '0'; wait for clk_period/2;
        clk <= '1'; wait for clk_period/2;
    end process;

    stim_proc: process begin
        res <= '1'; wait for 20 ns; res <= '0'; wait for 20 ns;

        -- TEST 1: Stato Base  $|0\rangle$ . In Fixed Point Q3.8 il valore 1.0 corrisponde a 256.
        alpha_in <= std_logic_vector(to_signed(256, 12)); beta_in <= (others => '0');
        wait for 40 ns;

        -- TEST 2: Stato Base  $|1\rangle$ 
        alpha_in <= (others => '0'); beta_in <= std_logic_vector(to_signed(256, 12));
```

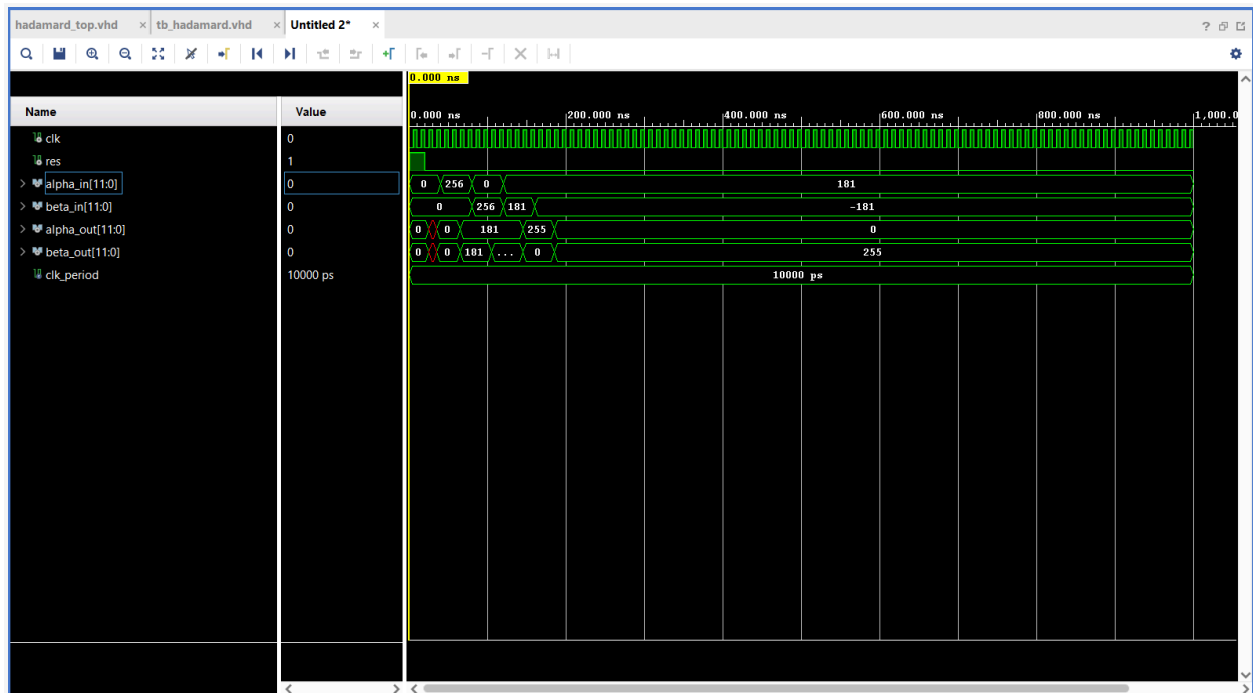
```

wait for 40 ns;

-- TEST 3: Sovrapposizione  $|+\rangle$ . L'operatore H dovrebbe collassarlo verso  $|0\rangle$ 
alpha_in <= std_logic_vector(to_signed(181, 12)); beta_in <=
std_logic_vector(to_signed(181, 12));
wait for 40 ns;

-- TEST 4: Sovrapposizione  $|-\rangle$ . L'operatore H dovrebbe collassarlo verso  $|1\rangle$ 
alpha_in <= std_logic_vector(to_signed(181, 12)); beta_in <=
std_logic_vector(to_signed(-181, 12));
wait;
end process;
end Behavioral;

```



Interpretazione dei risultati in simulazione:

Osservando le forme d'onda (tenendo conto della latenza architetturale di 2 colpi di clock introdotta dai registri di I/O), la logica è validata:

1. **Creazione della sovrapposizione:** L'ingresso di $|0\rangle$ (Test 1) produce coerentemente le ampiezze 181, 181 ($|+\rangle$). L'ingresso di $|1\rangle$ (Test 2) inverte correttamente la fase di β , producendo 181, -181 ($|-\rangle$).
2. **Reversibilità dell'Operatore:** Quando il sistema viene alimentato con gli stati di sovrapposizione (Test 3 e 4), il gate inverte il calcolo, ripristinando correttamente gli stati puri computazionali originari (valori prossimi a 256 e 0).

4. Risultati di Implementazione Hardware

Il design è stato sintetizzato e implementato con successo sulla Boolean Board (Spartan-7).

Risorsa	Valore Usato	Spiegazione Tecnica dell'Allocazione
Slice LUTs	13	Logica combinatoria pura per gestire le addizioni/sottrazioni incrociate.
Slice Registers	25	Flip-flop impiegati per rispettare la direttiva di sincronizzazione I/O.
DSPs	3	Blocchi DSP48E1 specializzati, che hanno permesso la <i>Register Absorption</i> (assorbendo parte dei registri di ingresso per ottimizzare i moltiplicatori).

Temporizzazione e Frequenza Massima Operativa ($F_{\max}$):

La Static Timing Analysis ha registrato un **Worst Negative Slack (WNS)** di **1.710 ns** rispetto a un vincolo di clock di 10 ns. Da questo valore si ricava la frequenza massima tollerabile dal circuito senza violazioni di setup, applicando la seguente formula:

$$F_{\max} = 1 / (T_{\text{clk}} - \text{WNS}) = 1 / (10 \text{ ns} - 1.710 \text{ ns}) = 1 / 8.29 \text{ ns} \approx 120.6 \text{ MHz}$$

Questo dimostra che la mappatura delle moltiplicazioni sui DSP ha ridotto drasticamente il cammino critico, permettendo performance elevatissime.

Dissipazione di Potenza:

Il report stima una *Total On-Chip Power* pari a soli **0.078 W**, quasi interamente dovuta alla potenza statica di *leakage* del chip, dimostrando che il modulo quantistico è estremamente parco nei consumi dinamici.

